

Лабораторная №11. Интеграция Arduino и Lichee RV Dock по SPI с датчиком BME280

Цель работы

Освоить совместную работу одноплатного компьютера Lichee RV Dock и микроконтроллера Arduino Uno в единой распределённой системе. Научиться считывать данные с датчика BME280 по I2C на Arduino, передавать их на Lichee через SPI-интерфейс и отображать на OLED-экране по I2C — тем самым объединяя знания и навыки, полученные в лабораторных №7 (SPI), №8 (I2C, OLED) и №10 (Arduino).

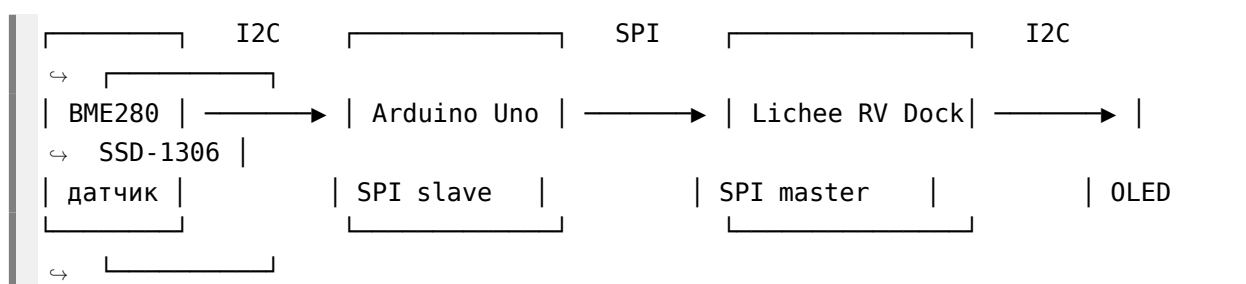
Оборудование и исходные данные

- Lichee RV Dock с настроенными SPI1 и I2C TWI2;
- Arduino Uno и USB-кабель;
- датчик BME280;
- OLED-дисплей SSD-1306;
- преобразователь логических уровней 5↔3,3 В;
- соединительные провода и макетная плата;
- компьютер с Arduino IDE, PulseView и логическим анализатором.

Ключевые понятия

Архитектура системы

Общая схема обмена данными:



Архитектура объединяет три интерфейса и два вычислительных устройства:

1. **Arduino → BME280 (I2C)** — чтение температуры, влажности и давления с датчика
2. **Arduino → Lichee (SPI)** — передача собранных данных на одноплатник; Arduino работает как SPI-ведомый (slave)
3. **Lichee → OLED (I2C)** — отображение полученных данных на экране SSD-1306 (лабораторная №8)

Передача по SPI выполняется серией из 12 однобайтовых транзакций — по одной на каждый байт трёх чисел формата float. Такой подход выбран

потому, что AVR-микроконтроллер ATmega328P в режиме SPI-slave имеет однобайтовый буфер передачи: при непрерывном тактировании нескольких байт (burst) shift register не перезагружается из буфера SPDR автоматически. Использование 12 отдельных транзакций (SS↓→1 байт→SS↑) гарантирует перезагрузку shift register из SPDR в начале каждой из них.

Внимание

Каждый байт передавайте отдельной SPI-транзакцией: опустить SS, передать ровно один байт, затем поднять SS. Не объединяйте 12 байт в одну транзакцию с постоянно активным SS, иначе описанный протокол Arduino-slave сформирует неверный ответ.

Датчик BME280 и библиотека GyverBME280

Определение

BME280 — цифровой датчик температуры, влажности и атмосферного давления производства Bosch Sensortec с интерфейсами I2C и SPI. В этой работе используется I2C.

Характеристики: - Диапазон температур: от −40 до +85°C, точность ±0.5°C - Диапазон влажности: от 0 до 100%, точность ±3% - Диапазон давления: от 300 до 1100 гПа, точность ±1 гПа - Напряжение питания: 1.8–3.6 В - I2C-адрес по умолчанию: **0x76** (вывод SDO замкнут на GND)

Опасность повреждения

В этой работе питайте BME280 только от 3.3V. Не подавайте 5 В на VCC датчика или линии I2C.

Подключение BME280 к Arduino:

BME280		Arduino Uno
-----		-----
VCC	→	3.3V
GND	→	GND
SDA	→	A4 (SDA)
SCL	→	A5 (SCL)

Библиотека GyverBME280

В отличие от громоздкой Adafruit BME280 (требуется Adafruit Sensor, Adafruit BusIO и пр.), GyverBME280 — лёгкая библиотека, занимающая минимум памяти и не требующая дополнительных зависимостей. Установка через менеджер библиотек Arduino IDE: «GyverBME280».

Проверка

Проверка библиотеки. Если библиотека не находится или её API отличается, убедитесь, что в менеджере Arduino IDE установлена имен-

но GyverBME280, и сверьтесь с примером из установленной версии.

Основные методы:

```
#include <GyverBME280.h>
GyverBME280 bme;

bme.begin();                // инициализация (I2C-адрес 0x76 по умолчанию)
bme.readTemperature();      // температура в °C (float)
bme.readHumidity();          // влажность в % (float)
bme.readPressure();          // давление в Па (float), для гПа разделить на 100
```

Простой пример чтения датчика:

```
#include <GyverBME280.h>
GyverBME280 bme;

void setup() {
  Serial.begin(115200);
  Wire.begin();
  if (bme.begin()) {
    Serial.println("BME280 OK");
  } else {
    Serial.println("ERROR: BME280 not found!");
  }
}

void loop() {
  Serial.print("T: "); Serial.print(bme.readTemperature(), 1);
  Serial.print(" C  H: "); Serial.print(bme.readHumidity(), 1);
  Serial.print(" %  P: "); Serial.println(bme.readPressure() / 100.0, 1);
  delay(1000);
}
```

SPI-обмен на Arduino в режиме ведомого устройства

Аппаратная организация SPI Arduino Uno построена на микроконтроллере ATmega328P, который имеет аппаратный модуль SPI. Контакты:

Функция	Пин Arduino Uno	В slave режиме
SS (Slave Select)	10	вход (LOW активирует слейв)
MOSI (Master Out Slave In)	11	вход
MISO (Master In Slave Out)	12	выход
SCK (Serial Clock)	13	вход

В режиме слейва Arduino не генерирует тактовый сигнал — SCK приходит от мастера (Lichee). Сигнал SS, опущенный в LOW, активирует слейв и сообщает ему о начале транзакции. По фронтам SCK происходит одновременный сдвиг битов: мастер выставляет бит на MOSI, слейв — на MISO.

Регистры SPI Работа модуля SPI определяется тремя регистрами:

Определение

SPCR (SPI Control Register) — регистр управления аппаратным модулем SPI.

Бит	Имя	Назначение
7	SPIE	SPI Interrupt Enable — разрешение прерывания по завершению передачи байта
6	SPE	SPI Enable — включение модуля SPI
5	DORD	Data Order: 0=MSB first, 1=LSB first
4	MSTR	Master/Slave Select: 0=Slave, 1=Master
3	CPOL	Clock Polarity: 0=SCK в покое LOW
2	CPHA	Clock Phase: 0=захват по переднему фронту
1-0	SPR	Clock Rate (только для мастера)

Для включения слейва с прерываниями (SPI mode 0):

```
SPCR = _BV(SPE) | _BV(SPIE); // 0b11000000: SPI вкл, прерывания вкл, mode 0
```

Определение

SPSR (SPI Status Register) — регистр состояния SPI. Бит SPIF устанавливается аппаратно по завершении передачи байта; для его сброса читают SPSR, а затем обращаются к SPDR.

Определение

SPDR (SPI Data Register) — регистр данных SPI. Записанный в него байт передаётся в следующей транзакции, а чтение возвращает последний принятый байт.

Прерывание SPI_STC_vect Когда байт полностью передан (8 тактов SCK), аппаратно устанавливается флаг SPIF, и если бит SPIE в SPCR равен 1, генерируется прерывание. Обработчик помечается макросом ISR(SPI_STC_vect).

Ключевой момент для слейва: **до начала следующей однобайтовой транзакции** (пока мастер не опустил SS в LOW) необходимо записать в

SPDR байт, который должен быть отправлен мастеру. Если мастер поднимет SS, а затем опустит снова — shift register загрузит содержимое SPDR заново.

Почему в работе используются отдельные транзакции?

При непрерывной передаче следующий ответный байт должен попасть в SPDR до начала его тактирования. Если обработчик прерывания не успеет подготовить данные, мастер получит устаревший или повторный байт. В этой работе 12 отдельных однобайтовых транзакций с поднятием SS между ними дают обработчику гарантированное время на подготовку следующего значения.

Базовый пример SPI-slave Минимальный скетч, отвечающий мастеру фиксированным байтом:

```
#include <SPI.h>

volatile byte g_response = 0xA5;

ISR(SPI_STC_vect) {
    (void)SPDR;          // чтение принятого байта (сброс SPIF)
    SPDR = g_response;    // подготовка ответа на следующую транзакцию
}

void setup() {
    Serial.begin(115200);
    pinMode(SS, INPUT_PULLUP);
    pinMode(SCK, INPUT);
    pinMode(MOSI, INPUT);
    pinMode(MISO, OUTPUT);
    SPCR = _BV(SPE) | _BV(SPIE);
    sei();
    SPDR = g_response;    // предзагрузка первого байта
    Serial.println("SPI slave ready");
}

void loop() {
    // прерывание делает всю работу
}
```

Пояснение: - `pinMode(SS, INPUT_PULLUP)` — встроенная подтяжка к HIGH, чтобы SS не плавало и не активировало слейв ложно - `SPCR = _BV(SPE) | _BV(SPIE)` — включение SPI в режиме слейва с прерываниями (MSTR=0 по умолчанию) - `sei()` — глобальное разрешение прерываний - `SPDR = g_response` — предзагрузка первого ответного байта до того, как мастер начнёт первую транзакцию. Без этого первый байт будет

случайным - В ISR: чтение SPDR (обязательно для сброса SPIF!), затем запись следующего ответного байта

SPI на Lichee RV Dock

Краткое напоминание: для работы SPI на Lichee необходимо:

1. Ядро с поддержкой SPI_SUN6I, DMA_SUN6I, SPI_SPIDEV
2. Device Tree с активированным &spi1 на пинах PD10-PD15
3. Устройство /dev/spidev1.0 (проверить: `ls -la /dev/spidev*`)

Подробная процедура настройки описана в лабораторной №7.

Проверка

Проверка устройства. Номер SPI-шины зависит от Device Tree и сборки ядра. Выполните `ls -la /dev/spidev*` и укажите найденные номера шины и устройства вместо значений из примера.

Пины SPI1 на Lichee RV Dock:

Сигнал	Пин Lichee	GPIO
CS	—	PD10
SCK	—	PD11
MOSI	—	PD12
MISO	—	PD13

Python-библиотека spidev:

```
import spidev

spi = spidev.SpiDev()
spi.open(bus=1, device=0)      # /dev/spidev1.0
spi.max_speed_hz = 1000000     # 1 МГц
spi.mode = 0                   # CPOL=0, CPHA=0

rx = spi.xfer2([0x00])[0]      # отправить один байт, получить ответ
spi.close()
```

`xfer2()` удерживает CS активным в пределах одного вызова и освобождает после его завершения. Поэтому 12 отдельных вызовов `xfer2([0x00])` формируют 12 однобайтовых транзакций.

I2C OLED SSD-1306 на Lichee (лабораторная №8)

Дисплей SSD-1306 — монохромный OLED 128×64 пикселя с I2C-интерфейсом. Подробная процедура настройки I2C на Lichee описана в лабораторной №8.

Подключение:

Опасность повреждения

OLED на I2C Lichee в этой работе питайте от 3.3V. Использовать 5 В можно только при подтверждённом для конкретного модуля согласовании питания и уровней линий SDA/SCL.

SSD1306		Lichee RV Dock
-----		-----
VCC	→	3.3V
GND	→	GND
SCL	→	TWI2_SCL (PE12)
SDA	→	TWI2_SDA (PE13)

I2C-адрес: 0x3C (стандартный для SSD-1306).

Проверка: `ls -la /dev/i2c-*`

Python-библиотека luma-oled:

```
from luma.core.interface.serial import i2c
from luma.oled.device import ssd1306
from luma.core.render import canvas
from PIL import ImageFont

serial = i2c(port=2, address=0x3C)
oled = ssd1306(serial)
font = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
↪ 14)

with canvas(oled) as draw:
    draw.text((0, 0), "Hello!", fill="white", font=font)
```

Согласование логических уровней

Arduino Uno работает от 5 В, Lichee RV Dock — от 3.3 В. Прямое соединение линий данных недопустимо: 5 В на входе Lichee может вывести GPIO-пины Allwinner D1 из строя.

Для согласования используется **логический преобразователь уровней** на макетной плате. Принцип работы: двунаправленный параллельный преобразователь на MOSFET-транзисторах, у которого каждая пара каналов имеет сторону HV (high voltage, 5 В) и LV (low voltage, 3.3 В).

Питание преобразователя:

Вывод преобразователя	Подключить к
HV	Arduino 5V
LV	Lichee 3.3V
GND	Общий GND (Arduino + Lichee)

Направления сигналов:

Сигнал	Направление	Подключение Arduino	Подключение Lichee
MOSI	Lichee → Arduino	MOSI (11) ↔ HV2	MOSI (PD12) ↔ LV2
SCK	Lichee → Arduino	SCK (13) ↔ HV3	SCK (PD11) ↔ LV3
SS/CS	Lichee → Arduino	SS (10) ↔ HV4	CS (PD10) ↔ LV4
MISO	Arduino → Lichee	MISO (12) ↔ HV1	MISO (PD13) ↔ LV1

Обратите внимание: для сигналов, идущих от Lichee к Arduino (MOSI, SCK, SS), задействована сторона LV как вход, HV как выход. Для сигнала MISO (Arduino → Lichee) — наоборот: HV как вход, LV как выход.

Полная схема соединений:

Опасность повреждения

До включения полной схемы установите преобразователь 5↔3,3 В между всеми линиями SPI Arduino и Lichee: HV подключите к Arduino 5 В, LV — к Lichee 3,3 В. GND Arduino, Lichee, преобразователя и периферии объедините в общий GND.

```

BME280          Arduino Uno
-----
VCC  —————→ 3.3V
GND  —————→ GND
SDA  —————→ A4
SCL  —————→ A5

```

```

Arduino Uno      Конвертер (HV)      Конвертер (LV)      Lichee RV Dock
-----
MISO (12) —————→ HV1      → LV1 —————→ MISO (PD13)
MOSI (11) ←———— HV2      ← LV2 —————→ MOSI (PD12)
SCK (13)  ←———— HV3      ← LV3 —————→ SCK (PD11)
SS (10)   ←———— HV4      ← LV4 —————→ CS (PD10)

```

Питание конвертера:

```

Arduino 5V  —————→ HV
Lichee 3.3V —————→ LV
Arduino GND —————→ GND ←— Lichee GND

```

```

SSD-1306 OLED    Lichee RV Dock
-----
VCC  —————→ 3.3V
GND  —————→ GND
SDA  —————→ TWI2_SDA (PE13)
SCL  —————→ TWI2_SCL (PE12)

```

Все GND (Arduino, Lichee, конвертер и периферия) должны быть соединены в общий GND — без этого сигналы не будут иметь общего уровня отсчёта и SPI не заработает.

Частая ошибка: путать MOSI и MISO при соединении. Правило простое — одноимённые пины соединяются друг с другом: MOSI↔MOSI, MISO↔MISO. НЕ перекрещивать!

Отладка логическим анализатором

Для проверки корректности SPI-обмена рекомендуется использовать логический анализатор (fx2lafw) и программу PulseView (лабораторная №9).

Опасность повреждения

Подключайте логический анализатор со стороны Lichee, после преобразователя уровней, где амплитуда сигналов равна 3,3 В. Обязательно соедините GND анализатора с общим GND схемы; не подавайте на его входы 5 В без подтверждения допустимого диапазона.

Проверка

Проверка обмена. Сначала добейтесь обмена тестовым скриптом, затем подтвердите в PulseView режим SPI и границы однобайтовых транзакций по линиям CS, SCK, MOSI и MISO.

Подключение анализатора (рекомендуется к пинам Lichee — все сигналы 3.3 В, безопасно для fx2lafw):

Канал анализатора	Пин Lichee	Сигнал
D0	PD10	CS
D1	PD11	SCK
D2	PD12	MOSI
D3	PD13	MISO
GND	GND	общий GND

Настройка PulseView:

1. Частота дискретизации: не менее **4 МГц** для SPI на 1 МГц.
2. Лимит семплов: 1-2М
3. Добавить декодер: Add protocol decoder → SPI
4. Привязать каналы: **CS#** = D0, **SCK** = D1, **MOSI** = D2, **MISO** = D3
5. Параметры декодера:

- CS# polarity: **Active low**
- CPOL: **0** (SCK в покое LOW)
- CPHA: **0** (захват по переднему/rising фронту)

- Bit order: **MSB first**
- Wordsize: **8 bits**

Что должно быть видно на экране:

При работающей системе каждые 2 секунды появляется серия из 12 SPI-транзакций. В каждой транзакции CS переходит в LOW, проходят 8 тактов SCK, затем CS возвращается в HIGH. При скорости 1 МГц передача восьми битов занимает около 8 мкс без учёта накладных расходов. На MOSI передаются нулевые байты, а на MISO — байты данных датчика.

Порядок выполнения

Подготовка оборудования

Убедитесь, что:

- **Lichee RV Dock:** настроен SPI (лабораторная №7), настроен I2C (лабораторная №8)
- Присутствуют файлы устройств:


```
ls -la /dev/spidev*
ls -la /dev/i2c-*
```
- Установлены Python-библиотеки:


```
apt-get install python3-spidev python3-module-luma-oled
```
- **Arduino Uno:** подключена к компьютеру, Arduino IDE установлена (apt-get install arduino)
- В менеджере библиотек Arduino IDE установлена **GyverBME280**

Проверка датчика BME280

Соберите часть схемы: подключите BME280 к Arduino (VCC→3.3V, GND→GND, SDA→A4, SCL→A5).

Загрузите тестовый скетч в Arduino:

```
#include <GyverBME280.h>
GyverBME280 bme;

void setup() {
  Serial.begin(115200);
  Wire.begin();

  if (bme.begin()) {
    Serial.println("BME280 OK (I2C 0x76)");
  } else {
    Serial.println("ERROR: BME280 not found!");
  }
}
```

```

    Serial.println("Check SDA(A4), SCL(A5), 3.3V, GND.");
    while (1);
  }
}

void loop() {
  float t = bme.readTemperature();
  float h = bme.readHumidity();
  float p = bme.readPressure() / 100.0;

  Serial.print("T: "); Serial.print(t, 1);
  Serial.print(" C  H: "); Serial.print(h, 1);
  Serial.print(" %  P: "); Serial.println(p, 1);

  delay(1000);
}

```

Откройте **Инструменты** → **Монитор порта** со скоростью 115200 бод.

Проверка

Ожидаемый результат. В мониторе порта появляются показания температуры, влажности и давления. При сообщении об ошибке проверьте SDA, SCL, питание 3,3 В и GND.

Проверка SPI-соединения

На этом шаге мы убедимся, что SPI-обмен между Arduino и Lichee работает. Arduino будет отвечать фиксированным байтом, Python на Lichee — читать его.

Скетч для Arduino

```

#include <SPI.h>

volatile bool g_done = false;

ISR(SPI_STC_vect) {
  (void)SPDR;           // читаем принятый байт
  SPDR = 0xA5;          // сразу готовим ответ на следующую транзакцию
  g_done = true;
}

void setup() {
  Serial.begin(115200);
  delay(500);
  pinMode(LED_BUILTIN, OUTPUT);

  pinMode(SS, INPUT_PULLUP);
  pinMode(SCK, INPUT);
}

```

```

pinMode(MOSI, INPUT);
pinMode(MISO, OUTPUT);

SPCR = _BV(SPE) | _BV(SPIE);
sei();

SPDR = 0xA5;          // предзагрузка первого байта

Serial.println("SPI slave ready. Sending 0xA5.");
}

void loop() {
  if (g_done) {
    g_done = false;

    static bool led = false;
    led = !led;
    digitalWrite(LED_BUILTIN, led);
  }
}

```

Загрузите скетч. Светодиод L на плате Arduino должен начать мигать при каждом SPI-запросе.

Тестовый Python-скрипт на Lichee Создайте файл test_spi.py:

```

#!/usr/bin/env python3
import spidev, time

spi = spidev.SpiDev()
spi.open(1, 0)          # /dev/spidev1.0
spi.max_speed_hz = 1000000
spi.mode = 0

print("SPI test: Ctrl+C to stop")
try:
    while True:
        rx = spi.xfer2([0x00])[0]
        print(f"RX: {rx:02X} (0x{rx:02X})")
        time.sleep(0.5)
except KeyboardInterrupt:
    spi.close()
    print("Done.")

```

Запустите python3 test_spi.py.

Проверка

Ожидаемый результат. В консоли повторяется RX: A5. Если выводится RX: FF, проверьте MISO, общий GND, направления каналов преобразователя уровней и его питание.

Отладка логическим анализатором Подключите логический анализатор к пинам Lichee (PD10=D0, PD11=D1, PD12=D2, PD13=D3, GND). Запустите захват в PulseView с частотой не менее 4 МГц одновременно с тестовым скриптом. Вы должны увидеть:

- CS (D0): короткие импульсы LOW (активный уровень)
- SCK (D1): 8 тактов на каждый CS-импульс
- MOSI (D2): все биты = 0
- MISO (D3): биты = 10100101 (0xA5)

Сборка полной схемы

Соберите схему полностью, руководствуясь таблицей соединений из подготовительного материала:

1. Подключите BME280 к Arduino (I2C)
2. Подключите Arduino к конвертеру уровней (SPI)
3. Подключите конвертер к Lichee (SPI)
4. Подайте питание на конвертер: HV = Arduino 5V, LV = Lichee 3.3V
5. Соедините GND всех устройств в общий GND
6. Подключите OLED SSD-1306 к 3.3V и I2C Lichee

На рисунке ниже приведен пример такого соединения:

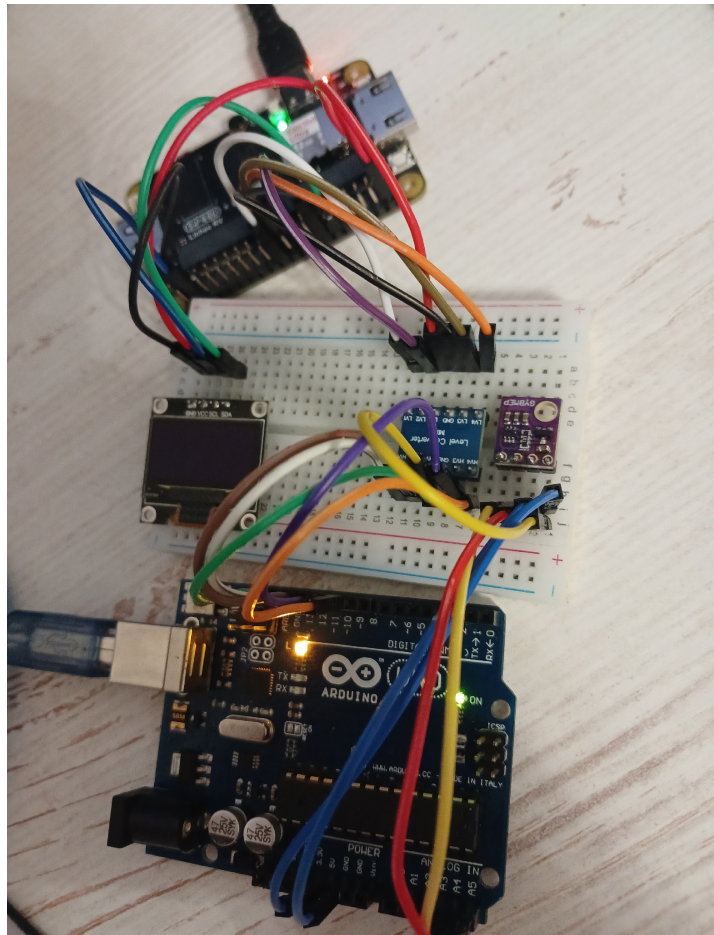


Рис. 1 – Пример полной схемы подключения

Итоговые программы

Итоговый скетч для Arduino Следующий листинг должен совпадать с каноническим примером `labs/examples/arduino_i2c_spi/arduino_spi_slave/ard`

```
#include <GyverBME280.h>
#include <SPI.h>
GyverBME280 bme;
volatile float      g_temp    = 0.0;
volatile float      g_hum     = 0.0;
volatile float      g_press   = 0.0;
volatile bool       g_ok      = false;
volatile byte       g_buf[12];
volatile byte       g_idx     = 0;
volatile bool       g_done    = false;
volatile unsigned int g_cnt    = 0;

void buf_load() {
    float t = g_temp;
    float h = g_hum;
    float p = g_press;
    noInterrupts();
    if (g_ok) {
        memcpy((void*)(g_buf + 0), &t, 4);
```

```

        memcpy((void*)(g_buf + 4), &h, 4);
        memcpy((void*)(g_buf + 8), &p, 4);
    }
    g_idx = 0;
    SPDR = g_buf[0];
    g_done = false;
    interrupts();
}
ISR(SPI_STC_vect) {
    (void)SPDR;
    g_idx++;
    if (g_idx < 12) {
        SPDR = g_buf[g_idx];
    } else {
        g_idx = 0;
        g_done = true;
    }
    g_cnt++;
}
void setup() {
    Serial.begin(115200);
    delay(500);
    pinMode(LED_BUILTIN, OUTPUT);
    Serial.println(F("Arduino SPI Slave + BME280"));
    Serial.println(F("====="));
    Wire.begin();
    if (bme.begin()) {
        g_ok = true;
        Serial.println(F("BME280 OK (I2C 0x76)"));
    } else {
        g_ok = false;
        Serial.println(F("ERROR: BME280 not found!"));
        Serial.println(F("Check SDA(A4), SCL(A5), 3.3V, GND."));
    }
    pinMode(SS, INPUT_PULLUP);
    pinMode(SCK, INPUT);
    pinMode(MOSI, INPUT);
    pinMode(MISO, OUTPUT);
    SPCR = _BV(SPE) | _BV(SPIE);
    sei();
    buf_load();
    Serial.println(F("SPI slave ready. Waiting for Lichee master..."));
    Serial.println(F("====="));
}
void loop() {
    static uint32_t last = 0;
    uint32_t now = millis();
    if (now - last >= 1000) {
        if (g_ok) {

```

```

//      noInterrupts();
      g_temp  = bme.readTemperature();
      g_hum   = bme.readHumidity();
      g_press = bme.readPressure() / 100.0;
//      interrupts();
    }
    Serial.print(F("T:"));
    Serial.print(g_temp, 1);
    Serial.print(F(" H:"));
    Serial.print(g_hum, 1);
    Serial.print(F(" P:"));
    Serial.print(g_press, 1);
    Serial.print(F(" cnt:"));
    Serial.println(g_cnt);
    last = now;
  }
  if (g_done) {
    buf_load();
    digitalWrite(LED_BUILTIN, !digitalRead(LED_BUILTIN));
  }
}

```

setup() инициализирует датчик и SPI, а loop() раз в секунду читает BME280. Обработчик SPI продвигает индекс буфера; после отправки 12 байт buf_load() атомарно обновляет буфер и предзагружает первый байт следующего набора. Светодиод L переключается после передачи полного набора данных.

Итоговый Python-скрипт для Lichee Следующий листинг должен совпадать с каноническим примером labs/examples/lichee_integration/lichee_spi_o

```

#!/usr/bin/env python3

import spidev
import struct
import time
from luma.core.interface.serial import i2c
from luma.oled.device import ssd1306
from luma.core.render import canvas
from PIL import ImageFont

SPI_BUS    = 1
SPI_DEVICE = 0
SPI_SPEED  = 1000000

I2C_PORT   = 2
I2C_ADDR   = 0x3C

```

```

UPDATE_INTERVAL = 2.0

try:
    font = ImageFont.truetype(
        "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf", 14)
except Exception:
    font = ImageFont.load_default()

def main():
    spi = spidev.SpiDev()
    spi.open(SPI_BUS, SPI_DEVICE)
    spi.max_speed_hz = SPI_SPEED
    spi.mode = 0
    print(f"SPI: /dev/spidev{SPI_BUS}.{SPI_DEVICE} @ {SPI_SPEED} Hz")

    serial = i2c(port=I2C_PORT, address=I2C_ADDR)
    oled = ssd1306(serial)
    oled.clear()
    print(f"OLED: I2C-{I2C_PORT}, addr 0x{I2C_ADDR:02X}")

    print("\n" + "=" * 44)
    print("SYSTEM RUNNING")
    print("  BME280 -> Arduino(I2C) -> Lichee(SPI) -> OLED(I2C)")
    print("  Press Ctrl+C to stop")
    print("=" * 44 + "\n")

    try:
        while True:
            rx = [spi.xfer2([0x00])[0] for _ in range(12)]
            data = bytes(rx)
            temp, hum, press = struct.unpack("<fff", data)

            print(f"T: {temp:6.1f} C    "
                  f"H: {hum:5.1f} %    "
                  f"P: {press:7.1f} hPa")

            with canvas(oled) as draw:
                draw.text((0, 0), f"Temp: {temp:.1f} C",
                           fill="white", font=font)
                draw.text((0, 20), f"Hum: {hum:.1f} %",
                           fill="white", font=font)
                draw.text((0, 40), f"Press: {press:.1f} hPa",
                           fill="white", font=font)

            time.sleep(UPDATE_INTERVAL)

    except KeyboardInterrupt:
        print("\nStopped by user.")
    finally:

```

```

oled.clear()
spi.close()
print("SPI closed, OLED cleared.")

if __name__ == "__main__":
    main()

```

Скрипт открывает SPI (bus=1, device=0, 1 МГц, режим 0) и I2C OLED (port=2, addr=0x3C). В бесконечном цикле он выполняет 12 однобайтовых SPI-транзакций, распаковывает данные как три числа float с порядком байтов от младшего к старшему (<fff) и выводит значения в консоль и на OLED. Интервал обновления составляет 2 секунды. При нажатии Ctrl+C скрипт очищает экран и закрывает SPI.

Запуск и проверка

1. **Загрузите скетч** `arduino_spi_slave.ino` на Arduino (плата: Arduino Uno, порт: `/dev/ttyUSB0` или аналогичный)
2. **Проверьте Serial-монитор** (115200 бод) — должны появиться сообщения об инициализации BME280 и SPI, а затем показания датчика и счётчик ISR (cnt:)

3. **Запустите Python-скрипт** на Lichee:

```
python3 lichee_spi_oled.py
```

4. **Убедитесь в корректной работе:**

- В консоли Lichee: строки T: ... C H: ... % P: ... hPa с реальными данными
- На OLED-экране: три строки с температурой, влажностью и давлением
- На Arduino: светодиод L мигает, счётчик ISR растёт

5. **Проверьте логическим анализатором (опционально):** 12 CS-импульсов с 8 тактами SCK каждый, MOSI=0, MISO меняется

Проверка

Ожидаемый результат. В консоли Lichee и на OLED отображаются реалистичные показания BME280, светодиод L на Arduino переключается после обмена, а счётчик ISR растёт.

Задание

Ознакомьтесь с ключевыми понятиями и выполните следующие подзадачи.

1. **Проверить работу датчика.** Подключите BME280 к Arduino по I2C. Напишите скетч, читающий температуру, влажность и давление, и убедитесь в корректности показаний через Serial-монитор.
2. **Проверить SPI-соединение.** Соберите SPI-цепь (Arduino → конвертер уровней → Lichee). Напишите скетч, отвечающий фиксированным байтом 0xA5, и Python-скрипт на Lichee, читающий этот байт. Добейтесь уверенного приёма. Проверьте сигналы логическим анализатором в PulseView (опционально).
3. **Собрать полную схему.** Руководствуясь таблицей соединений, соберите схему целиком: BME280→Arduino, Arduino→Конвертер→Lichee (SPI), OLED→Lichee (I2C). Обеспечьте общий GND.
4. **Протестировать финальную программу.** Загрузите итоговый скетч в Arduino и запустите Python-скрипт на Lichee. На OLED-экране должны отображаться температура, влажность и давление с датчика BME280.
5. **Ответить на контрольные вопросы (письменно, в отчёте).**
6. **Ответить на вопросы преподавателя.**
7. **Продемонстрировать работу преподавателю.**

Контрольные вопросы

1. Почему для подключения BME280 к Arduino используется I2C, а не SPI?
2. Какую роль выполняет сигнал SS в SPI и почему на Arduino он настроен как INPUT_PULLUP?
3. Зачем в скетче выполняется предзагрузка SPDR = g_buf[0] до начала SPI-транзакций?
4. Почему для передачи 12 байт используется 12 отдельных однобайтовых транзакций, а не одна 12-байтовая? Что произойдёт при пакетной передаче?
5. Для чего нужен логический преобразователь уровней? Какие сигналы проходят через него в направлении HV→LV, а какие — LV→HV?
6. Как `struct.unpack("<fff", data)` интерпретирует 12 байт, полученных по SPI? Что означает префикс `<`?
7. Какие регистры ATmega328P управляют работой SPI в режиме ведомого устройства? Какие биты необходимо установить?
8. Как отдельные вызовы `spi.xfer2()` формируют однобайтовые SPI-транзакции?

9. Как проверить корректность SPI-обмена с помощью логического анализатора? Какие сигналы должны быть видны на каждом канале?
10. Почему важно соединить GND Arduino, Lichee и конвертера в общий GND? Что произойдёт, если этого не сделать?

Требования к отчёту

Отчёт должен содержать:

- цель работы;
- схему или фотографию полной аппаратной конфигурации;
- результаты отдельных проверок BME280 и SPI;
- исходные тексты итоговых программ или ссылки на использованные канонические примеры;
- снимок OLED с показаниями датчика;
- результаты проверки логическим анализатором, если она выполнялась;
- ответы на контрольные вопросы;
- краткий вывод.

Отправьте отчёт в согласованном формате до установленного срока.